

SEVA SANKALP: TECHNICAL ARCHITECTURE, ALGORITHM SPECIFICATIONS, WORKFLOWS & VIVA DEFENSE DOSSIER

Comprehensive Technical Master Notes & Oral Examination Guide for Project Reviewers & Evaluators

Project Title	Seva Sankalp — Hyperlocal Peer-to-Peer Donation & Statutory NGO Verification Platform
Domain & Focus	Web Engineering, Geospatial Computing, Anti-Fraud Systems, Cryptographic Verification, Civic Tech
Team Members	Neyigapula Meghana (21331A05D8), Sandaka Anish Nihaal (21331A05G5), Mudunuri Sruthi (21331A05E2), Praveen Sahu (21331A05G0)
Project Guide	Mr. S. Palavelli, M.Tech., (Ph.D.), Assistant Professor, Department of CSE
Institution	Maharaj Vijayaram Gajapathi Raj (MVGR) College of Engineering (Autonomous), Vizianagaram
Key Technologies	Python 3.10+, Flask, SQLAlchemy, SQLite/PostgreSQL, Leaflet.js, OpenStreetMap, Bcrypt, Jinja2, Bootstrap 5

1. ARCHITECTURAL PARADIGM & ENGINEERING DESIGN

1.1 Model-View-Controller (MVC) Pattern with Modular Flask Blueprints

Seva Sankalp is engineered using the **Model-View-Controller (MVC)** architectural paradigm, implemented via Python Flask's native **Blueprint** mechanism. This design decouples presentation, business processing, and persistence into distinct layers:

- **Model Layer** (`app/models.py`): Defines the object-relational data contracts using SQLAlchemy ORM. Models encapsulate business rules, constraints (e.g., `'unique_user_ngo_like'`), relationship cascades (`'cascade='all, delete-orphan'`), password hashing algorithms, and dynamic state evaluation methods.
- **View Layer** (`app/templates/`, `app/static/`): Utilizes the Jinja2 templating engine combined with Bootstrap 5, Leaflet.js, and custom responsive CSS. Views are strictly decoupled from database queries, receiving contextual data dictionaries and rendering semantic, accessible, mobile-optimized HTML5.
- **Controller Layer** (`app/core/*_routes.py`): Decomposed into 8 isolated Blueprints (`'auth_routes'`, `'resource_routes'`, `'request_routes'`, `'search_routes'`, `'profile_routes'`, `'admin_routes'`, `'points_routes'`, `'history_routes'`). Controllers handle HTTP request dispatching, input validation, cryptographic session handling, and coordination between database models and external services.

1.2 Application Factory Pattern & Configuration Decoupling

The application initialization follows the **Application Factory Pattern** (`create_app()` in `app/__init__.py`). Instead of binding the application instance globally, the factory dynamically configures extensions (`db`, `login_manager`, `mail`) based on runtime environments (`DevelopmentConfig`, `ProductionConfig`,

TestingConfig). This guarantees zero state leakage across test suites and enables multiple worker processes in production WSGI servers.

1.3 Architectural Trade-Off Analysis: Monolithic MVC vs. Microservices

During architectural formulation, a monolithic MVC design was deliberately chosen over a distributed microservices architecture based on concrete engineering metrics:

Evaluation Metric	Monolithic MVC (Implemented)	Distributed Microservices (Rejected)
Network Latency	In-memory function calls (< 1 ms overhead); zero internal HTTP serialization.	Inter-service HTTP/gRPC overhead (15-50 ms per request hop across services).
ACID Data Integrity	Single database engine guarantees immediate consistency via atomic SQL transactions.	Requires distributed 2-Phase Commit (2PC) or Saga patterns; eventual consistency risks.
Deployment Overhead	Single WSGI container deployment on PythonAnywhere / Gunicorn; minimal DevOps cost.	Requires container orchestrators (Kubernetes/Docker), API gateways, service meshes.
State Management	Flask-Login centralized session handling with secure HttpOnly cookie tokens.	Requires distributed JWT token stores, OAuth authorization servers, and Redis sync.
Development Velocity	Unified codebase, instant schema migrations, straightforward debugging and profiling.	Complex multi-repo management, network failure modes, RPC versioning conflicts.

Key Viva Takeaway (Architecture)

When judges ask 'Why did you not use Microservices?', state: 'For a community-scale civic application, microservices introduce distributed network latency, data synchronization risks, and heavy infrastructure costs without adding value. Our modular Flask Blueprint architecture delivers identical clean separation of concerns with the ultra-low latency, immediate ACID transactional consistency, and zero DevOps overhead of a production-grade monolithic deployment.'

2. COMPLETE END-TO-END TECHNOLOGY STACK (DEEP DIVE)

Every tier of Seva Sankalp is built on open-source, industry-standard, production-proven technologies selected for speed, security, and developer ergonomics:

Layer / Domain	Technology & Version	Core Role in Seva Sankalp	Engineering Justification over Alternatives
Frontend UI	HTML5 & CSS3	Semantic layouts, responsive flex/grid, dark mode	Native browser performance, accessibility (WAI-ARIA), no heavy bundle build steps.
Frontend Styling	Bootstrap 5.3	Responsive grid, modals, alert banners, badges	Rapid responsive prototyping; avoids React/Vue build complexity for SSR applications.

Layer / Domain	Technology & Version	Core Role in Seva Sankalp	Engineering Justification over Alternatives
Geospatial GIS	Leaflet.js 1.9.4	Interactive map rendering, custom marker popups	Lightweight (42 KB vs 300 KB Google Maps), zero commercial API billing, privacy-preserving.
Map Tiles	OpenStreetMap (OSM)	Tile layer imagery via OpenStreetMap Carto	Free community-driven tile server; eliminates dependency on Google Cloud billing quotas.
Iconography	FontAwesome 6.4	Visual cues: heart like, pin, badges, status	Scalable vector icons with clean semantic class integration across UI components.
Client Scripting	Vanilla JavaScript (ES6+)	Asynchronous Fetch API for likes, chat, map sync	Native browser execution without NPM/Node compilation dependencies; zero bundle bloat.
Backend Runtime	Python 3.10 / 3.13	Core computational engine and business logic	Robust standard library ('math', 'datetime', 're'), clean syntax, excellent ORM support.
Web Framework	Flask 3.0.x (WSGI)	Routing, request handling, Blueprint modularity	Microframework flexibility; avoids Django's rigid monolithic boilerplate and ORM lock-in.
Templating	Jinja2 3.1.x	Server-Side Rendering (SSR) & XSS protection	Automatic context-aware HTML escaping, template inheritance, lightning-fast rendering.
Database ORM	SQLAlchemy 2.0.x	Object-Relational Mapping & query compilation	Type-safe database abstraction, parameterized queries (SQLi immune), cascade automation.
Database Engine	SQLite / PostgreSQL	ACID persistence, relational data integrity	SQLite for lightweight zero-config MVP; fully schema-compatible with PostgreSQL for cloud.
Authentication	Flask-Login 0.6.x	User session state machine & route protection	Secure cookie-based authentication, user loader abstraction, automatic session teardown.
Password Security	Bcrypt 4.x	Blowfish cryptographic hashing with adaptive salt	Computationally resistant to GPU/ASIC brute-force cracking, unlike MD5 or raw SHA-256.
Form Security	Flask-WTF / CSRFProtect	Cryptographic token injection on POST requests	Stops Cross-Site Request Forgery via signed HMAC session tokens on all stateful forms.
Web Server Gateway	Werkzeug WSGI	HTTP request dispatching & secure file naming	'secure_filename()' sanitization stops directory traversal attacks ('../etc/passwd').

Layer / Domain	Technology & Version	Core Role in Seva Sankalp	Engineering Justification over Alternatives
Production WSGI	PythonAnywhere / Gunicorn	Multi-worker concurrent request processing	WSGI-compliant process management, automatic recycling of worker processes.

3. CORE ALGORITHMS & MATHEMATICAL FORMULATIONS

3.1 Algorithm 1: Spherical Geodesic Haversine Distance Search

Hyperlocal resource discovery requires calculating the precise great-circle distance between two geographic coordinates on Earth given by latitude (ϕ) and longitude (λ).

- **Mathematical Formulation:** Earth is an oblate spheroid, but for short-to-medium regional distances (1 to 50 km), modeling it as a sphere of mean radius $R = 6,371.0 \text{ km}$ yields accuracy within 0.3%. The Haversine trigonometric formulation is:

The Haversine Equations

- Latitude Delta: $\Delta\phi = \phi_2 - \phi_1$ (converted to radians: $\text{deg} \times \pi / 180$)
- Longitude Delta: $\Delta\lambda = \lambda_2 - \lambda_1$ (converted to radians: $\text{deg} \times \pi / 180$)
- Square of Half Chord Length:

$$a = \sin^2(\Delta\phi / 2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda / 2)$$
- Angular Distance in Radians:

$$c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1 - a})$$
- Great-Circle Distance:

$$d = R \cdot c \quad (\text{where } R = 6,371.0 \text{ km})$$

- **Why Not Flat Euclidean Distance?:** A naive Pythagorean flat approximation $d = \sqrt{(\Delta x)^2 + (\Delta y)^2}$ produces catastrophic errors. On Earth, lines of longitude converge toward the poles ($1^\circ \text{ lon} = 111.32 \text{ km} \cdot \cos(\phi)$), whereas latitude lines remain constant ($1^\circ \text{ lat} \approx 111 \text{ km}$). A flat calculation treats degrees as square, distorting distances by 15% to 40% even over modest city distances.

- **Algorithmic Implementation in `app/core/search\_routes.py`:**

```
import math

def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0 # Earth's mean radius in kilometers
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat / 2) ** 2 +
         math.cos(math.radians(lat1)) * math.cos(math.radians(lat2)) * math.sin(dlon / 2) ** 2)
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return R * c # Returns great-circle distance in km
```

- **Computational Complexity & Spatial Optimization:** Evaluating N active resources in memory requires $O(N)$ trigonometric calculations. For scaling beyond 10,000 resources, Seva Sankalp supports **Bounding Box Pre-Filtering**: compute $[\text{lat} \pm \Delta \text{lat}, \text{lon} \pm \Delta \text{lon}]$ where $\Delta \text{lat} = \frac{\text{radius}}{111.0}$ and $\Delta \text{lon} = \frac{\text{radius}}{111.0 \cdot \cos(\text{lat})}$.

$\cos(\text{lat})$). The SQL database uses standard B-tree index scans on latitude and longitude in $O(\log N)$ time, evaluating the exact Haversine equation only on the candidate subset.

3.2 Algorithm 2: Anti-Flipping & Anti-Hoarding Sliding-Window Cooldown

Free redistribution platforms frequently suffer from predatory resellers and hoarders who collect donated electronics and appliances to sell on commercial marketplaces (e.g., OLX, Cashify). Seva Sankalp implements a multi-tier programmatic barrier:

- **1. Anti-Hoarding Concurrent Cap:** A recipient cannot create a new request if they already have ≥ 3 active requests with status 'Pending' across all platform resources: $\text{COUNT}(\text{Request}_{\text{status='Pending'}}, \text{receiver}=\text{current_user}) < 3$. Time complexity: $O(1)$ indexed count.
- **2. Category Lock (Active Request Mutual Exclusion):** A user cannot have more than 1 active request ('Pending' or 'Accepted') in the 'Electronics' or 'Household' categories concurrently.
- **3. Category-Specific Cooldown Windows:** When an item in a high-value category is marked 'Fulfilled', a dynamic cooldown timer is stamped against the recipient's user ID. If $T_{\text{current}} - T_{\text{fulfilled}} < \Delta t_{\text{category}}$, subsequent requests for that category are strictly rejected:

Category	Cooldown Window (Δt)	Rationale & Resale Vulnerability	Rule Enforcement Condition
Electronics	365 Days (12 Months)	Laptops, phones, tablets have high cash-out value; 1 year ensures genuine academic/work use.	IF (now - last_fulfilled.updated_at).days < 365 THEN REJECT
Household	180 Days (6 Months)	Appliances (fans, stoves, blenders) have durable utility; 6 months prevents household hoarding.	IF (now - last_fulfilled.updated_at).days < 180 THEN REJECT
Books & Clothes	0 Days (No Cooldown)	Low resale value; educational and clothing needs recur frequently throughout the year.	Allowed immediately subject to the global 3-request pending cap.

- **4. Mandatory Impact Proof Gating:** Before submitting any new request, the platform queries whether the recipient has fulfilled requests lacking an uploaded receipt photo: $\text{DonationHistory.receipt_photo} = \text{None}$. If unproven transactions exist, all new requests are locked until the user uploads physical photo proof of impact.

3.3 Algorithm 3: Fraud-Proof Karma Ledger & Anti-Gaming Engine

Gamification badges and Karma points encourage community participation but are vulnerable to Sybil attacks (users creating fake dummy accounts to donate fictitious items back and forth). Seva Sankalp eliminates point gaming through an idempotent, multi-check audit ledger:

- **1. Client IP Collision Detection:** Upon transaction fulfillment, the server retrieves 'current_user.last_login_ip' and 'receiver.last_login_ip'. If $\text{IP}_{\text{donor}} == \text{IP}_{\text{receiver}}$ (and $\text{IP} \neq '127.0.0.1'$), points awarded are reset to **0**, and a fraud audit log is generated. The physical handover proceeds to avoid penalizing legitimate users, but no gaming reward is granted.
- **2. Sliding 7-Day Velocity Cap (Max 300 Points):** To prevent artificial point surges, the engine calculates the sum of all points earned by the user over the trailing 7 days: $\sum_{t \in T_{\{7\text{days}\}}} \text{amount}$. If $\text{sum} + \text{points_awarded} > 300$, points awarded are clamped to 0.

- **3. Tier Level Calculation Function:** Badges are programmatically updated following the fulfillment transaction:

```
# app/core/request_routes.py - Karma Points Allocation
points_awarded = 50
if current_user.last_login_ip == receiver.last_login_ip and current_user.last_login_ip !=
'127.0.0.1':
    points_awarded = 0 # Same-network collision detected

if points_awarded > 0:
    seven_days_ago = datetime.utcnow() - timedelta(days=7)
    recent_points = db.session.query(func.sum(PointsTransaction.amount)).filter(
        PointsTransaction.user_id == current_user.id,
        PointsTransaction.created_at >= seven_days_ago).scalar() or 0
    if recent_points + points_awarded > 300:
        points_awarded = 0 # Velocity cap exceeded

if points_awarded > 0:
    current_user.points_balance += points_awarded
    if current_user.points_balance >= 5000: current_user.badge_level = 'Platinum'
    elif current_user.points_balance >= 1000: current_user.badge_level = 'Gold'
    elif current_user.points_balance >= 500: current_user.badge_level = 'Silver'
    elif current_user.points_balance >= 100: current_user.badge_level = 'Bronze'
```

3.4 Algorithm 4: Statutory NGO & DigiLocker e-KYC Verification Pipeline

To prevent fly-by-night fraudulent charities from collecting public goods and monetary donations, non-profits undergo strict administrative auditing against statutory government registries:

- **NITI Aayog NGO Darpan ID Validation:** Verified against the statutory central portal format `^[A-Z]{2}\d{4}\d{7}$` (e.g., `AP/2021/0284719`), representing state code, registration year, and 7-digit sequential unique identity.
- **Income Tax Department Organization PAN:** Verified against India's statutory 10-character alphanumeric structure `^[A-Z]{5}[0-9]{4}[A-Z]{1}$` where the 4th character must be 'C' (Company), 'T' (Trust), or 'A' (Association of Persons).
- **DigiLocker e-KYC Integration Flow:** DigiLocker allows users to prove credentials (Aadhaar, MeeSeva Income Certificates) without sharing raw identity numbers. The simulated/production pipeline uses OAuth 2.0 PKCE with API Setu to obtain a cryptographically signed XML document, validating eligibility without exposing unredacted records.
- **Confidential EWS / Low-Income Gating:** When a donor marks a resource as `requires_income_proof = True`, only recipients with `income_verification_status == 'approved'` can submit requests. Crucially, donors never see the recipient's private financial certificate; the system displays an official verified badge, preserving beneficiary dignity while satisfying donor intent.

3.5 Algorithm 5: Bcrypt Cryptographic Key Derivation & Password Storage

User passwords are never stored in plaintext or reversible encryption. Seva Sankalp uses the **Bcrypt** cryptographic password-hashing function based on the Blowfish block cipher:

- **Salt Generation:** A cryptographically secure pseudo-random 128-bit salt is generated per user via ``bcrypt.gensalt()``. This completely defeats pre-computed rainbow table attacks.
- **Adaptive Cost Factor (Work Factor = 12):** The algorithm executes $2^{12} = 4,096$ iterations of the Blowfish key schedule. Even if an attacker gains read access to the database, cracking a single 8-character password via brute-force requires prohibitive GPU processing time, whereas legitimate login verification takes only ~ 100 ms on the web server.

- **Timing Attack Defense:** Password verification invokes ``bcrypt.checkpw()``, which performs constant-time string comparison, neutralizing side-channel timing analysis.

4. SYSTEM TECHNICAL WORKFLOWS & STATE MACHINES

4.1 Workflow 1: User Onboarding & DigiLocker e-KYC Flow

- **Step 1 (Form Submission):** User fills registration form with email, password, and phone number.
- **Step 2 (DigiLocker Verification Option):** User can click 'Register with DigiLocker'. The system initiates an OAuth 2.0 redirection handshake, requests citizen identity scopes, and fetches verified profile metadata.
- **Step 3 (Password Hashing):** Controller invokes ``user.set_password(pwd)``, generating a 128-bit salt and 60-character modular crypt format string (``$2b$12$...``).
- **Step 4 (OTP Verification):** A 6-digit cryptographic OTP is generated and cached with a 10-minute expiry timestamp. User confirms email before account activation.

4.2 Workflow 2: Donor Resource Publishing & Geocoding

- **Step 1 (Input & Image Upload):** Donor enters title, description, category (``Electronics``, ``Books``, ``Clothing``, ``Household``), condition, and uploads an image.
- **Step 2 (MIME Validation & Sanitization):** Controller uses ``werkzeug.utils.secure_filename()`` to strip path separators, validates file extension against an allowlist (``png``, ``jpg``, ``jpeg``, ``webp``), and writes to ``static/uploads/resources/`` with a unique timestamped filename.
- **Step 3 (Geocoding & Persistence):** Coordinates (``location_lat``, ``location_lng``) and address are captured via browser Geolocation API or map pin click and committed to the ``resources`` table with status ``Available``.

4.3 Workflow 3: Hyperlocal Search & Distance Filtering

- **Step 1 (Client Coordinate Capture):** Browser obtains user GPS via ``navigator.geolocation.getCurrentPosition()`` or user enters search radius (1–50 km).
- **Step 2 (AJAX Request):** Frontend dispatches ``GET /search/api/resources?lat=18.11&lng=83.40&radius=10&category=Electronics``.
- **Step 3 (Server-Side Haversine Calculation):** Flask queries all ``status='Available'`` resources, computes Haversine distance, discards items where ``distance > radius``, and sorts by ascending distance.
- **Step 4 (Leaflet Map Render):** Client receives JSON, clears existing map layers, and plots custom interactive SVG markers. Clicking a marker displays item thumbnail, distance badge, and a 'Request Item' action button.

4.4 Workflow 4: Request Pipeline, Verification & Anti-Fraud Execution

When a recipient clicks 'Send Request', the system executes a sequential 6-gate safety validation before creating the transaction:

- **Gate 1 (Account Status):** Verify ``current_user.verification_status == 'approved'`` and ``current_user.is_ngo == False``.
- **Gate 2 (Pending Impact Proof):** Query ``DonationHistory`` for unproven past fulfillments; block if receipts are missing.
- **Gate 3 (Anti-Hoarding Cap):** Verify active pending requests count is strictly ``< 3``.
- **Gate 4 (EWS / Income Gate):** If ``resource.requires_income_proof``, verify ``current_user.income_verification_status == 'approved'``.
- **Gate 5 (Electronics Cooldown):** If category is ``Electronics``, ensure 0 active electronics requests and ``>= 365`` days since last fulfilled electronics item.

- **Gate 6 (Household Cooldown):** If category is 'Household', ensure 0 active household requests and \$ge 180\$ days since last fulfilled household item.
- **Commit:** Create record in 'requests' table with status 'Pending'; trigger email notification to donor.

4.5 Workflow 5: Handover Coordination & In-App Messaging

- **Step 1 (Donor Decision):** Donor views recipient profile, trust score, and badge. Donor clicks 'Accept' or 'Reject'.
- **Step 2 (Chat Channel Initialization):** When status transitions to 'Accepted', an in-app chat channel unlocks ('/request/<id>/messages').
- **Step 3 (Safe Coordination):** Donor and recipient exchange logistics details. Crucially, private phone numbers and personal home addresses are never exposed publicly, preventing harassment.

4.6 Workflow 6: Fulfillment, Impact Proof & Karma Audit

- **Step 1 (Handover Completion):** After physical exchange, donor clicks 'Mark as Fulfilled'. Both resource and request transition to 'Fulfilled'.
- **Step 2 (Anti-Gaming Check):** Server inspects 'last_login_ip' of both parties and checks trailing 7-day points velocity.
- **Step 3 (Ledger Record):** If legitimate, 50 Karma points are appended to 'points_transactions' and 'user.points_balance' is incremented.
- **Step 4 (Mandatory Impact Receipt):** Recipient is required to upload a photo of the item in active use, closing the accountability loop.

4.7 Workflow 7: Statutory NGO Support & Community Endorsements (❤️ Likes)

- **Step 1 (Registration):** NGO submits Darpan ID, PAN number, registration certificate, and UPI VPA ID ('ngo@upi').
- **Step 2 (Admin Review):** Admin audits statutory credentials and approves the NGO, publishing it to '/support-ngos'.
- **Step 3 (Direct UPI Support):** Citizens scan the dynamically rendered UPI QR code ('upi://pay?pa=...&pn=...') to donate directly to the NGO's bank account with 0 platform cuts.
- **Step 4 (❤️ Community Like Toggle):** Authenticated users can click the heart button. JavaScript dispatches 'POST /like-ngo/<id>'. The controller toggles the 'NGOLike' record idempotently and returns '{ liked: true/false, likes_count: N }', updating the UI instantly without page reload.

5. EXHAUSTIVE VIVA VOCE & TECHNICAL DEFENSE Q&A (45 MASTER QUESTIONS)

Below are 45 rigorous technical questions categorized across software engineering disciplines, complete with authoritative model answers tailored for judge evaluations:

5.1 Category 1: System Architecture & Framework Decisions

Q1: Why did you choose Flask over Django or Node.js/Express?

- **Granular Architectural Control:** Django is an opinionated framework that imposes rigid conventions, heavyweight ORM defaults, and built-in admin overhead that was unnecessary for our custom anti-fraud and geolocation workflows.
- **Microframework Lightweight Footprint:** Flask provides a lightweight WSGI core allowing us to select precisely the extensions we need (SQLAlchemy 2.0, Flask-Login, Bcrypt) without bundle bloat.

- **Native Python Scientific Ecosystem:** Python gives us direct access to mathematical libraries (`math`, `datetime`, future AI integrations) which Node.js lacks natively.

Q2: What is the Application Factory Pattern and why did you use it?

- **Explanation:** Instead of instantiating `app = Flask(__name__)` globally at the module level, we define a function `create_app(config_name)` inside `app/__init__.py` that instantiates, configures, and binds extensions to the app dynamically.
- **Benefits:** 1) Prevents circular imports between route Blueprints and models. 2) Allows isolated testing with different configurations (e.g., in-memory SQLite for tests vs file-based SQLite in production). 3) Enables WSGI servers to spawn clean worker processes.

Q3: How do Flask Blueprints work under the hood?

- **Concept:** A Blueprint is a recording mechanism for routes and handlers that can be registered onto a Flask application instance. It represents a logical slice of the application.
- **Implementation in Seva Sankalp:** We created 8 independent Blueprints (`auth_bp`, `resource_bp`, `request_bp`, etc.) each with its own URL prefix (e.g., `/request`, `/search`). When `app.register_blueprint()` is called in the factory, Flask attaches these routes to the central Werkzeug URL map.

Q4: How is state managed across requests in your web application?

- **Stateless HTTP with Cryptographic Cookies:** HTTP is inherently stateless. Flask-Login uses cryptographically signed session cookies stored in the client browser.
- **Security Attributes:** The session cookie contains an encrypted user identifier. It is marked `HttpOnly` (inaccessible to JavaScript, stopping XSS cookie theft) and `SameSite=Lax` (stopping CSRF cookie leakage).
- **User Loader:** On every incoming request, `@login_manager.user_loader` extracts the user ID from the session and retrieves the active `User` model from the database into `flask_login.current_user`.

Q5: What happens when two users try to request the same item at the exact same millisecond (Race Condition)?

- **Current Concurrency Control:** SQLite serializes write transactions using a database-level lock. Only one `INSERT` into `requests` executes first.
- **Application Level Mutual Exclusion:** The controller executes `already_requested = DonationRequest.query.filter_by(resource_id=..., receiver_id=...).first()`. Once the donor accepts one request, the resource transitions to `status = 'Requested'` or `'Fulfilled'`, automatically invalidating subsequent acceptance actions.
- **Production Database Lock:** In PostgreSQL, this is handled via `'SELECT FOR UPDATE'` or a composite unique constraint on `(resource_id, receiver_id)` enforcing database-level atomicity.

Q6: What is the role of Werkzeug in your application?

- **WSGI Foundation:** Werkzeug is the comprehensive WSGI web application library powering Flask underneath. It parses HTTP headers, cookies, URL routing rules, and multi-part form file data.
- **Security Utility:** We explicitly use `werkzeug.utils.secure_filename()` on uploaded documents to strip dangerous characters like `../` and null bytes, preventing directory traversal exploits.

Q7: Why did you implement Server-Side Rendering (SSR) with Jinja2 instead of a React/Vue SPA?

- **Zero Bundle Overhead:** Single-Page Applications require multi-megabyte JavaScript bundles, client-side hydration delays, and complex Webpack/Vite build toolchains that slow down low-bandwidth mobile users in rural areas.
- **Instant First Contentful Paint (FCP):** Jinja2 compiles semantic HTML directly on the server in ~15 ms, delivering immediate rendering on budget smartphones.

- **Enhanced Security:** CSRF tokens are injected directly into server-rendered forms, avoiding token storage vulnerabilities in browser `localStorage`.

5.2 Category 2: Core Algorithms & Mathematical Formulations

Q8: Derive the Haversine formula and explain why it is preferred over the Spherical Law of Cosines.

- **Derivation Basis:** The Haversine formula calculates the angular distance between points on a sphere from their latitudes and longitudes using half-angles: $\text{haversin}(\theta) = \sin^2(\theta / 2) = \frac{1 - \cos \theta}{2}$.
- **Numerical Precision at Small Distances:** The classic Spherical Law of Cosines uses $\cos(c) = \sin(\phi_1)\sin(\phi_2) + \cos(\phi_1)\cos(\phi_2)\cos(\Delta \lambda)$. For nearby points (e.g., 500 meters apart), $\cos(c)$ is extremely close to 1.0 (e.g., 0.99999998). Floating-point roundoff error in computers causes severe precision loss (catastrophic cancellation). Haversine avoids this by working with $\sin^2(\Delta / 2)$, maintaining floating-point accuracy down to centimeters.

Q9: What is the time complexity of your geospatial search?

- **Current Complexity:** $O(N)$ where N is the number of available resources. For each item, the Haversine function evaluates 7 trigonometric operations in memory.
- **Practical Benchmark:** In Python, 1,000 resources are evaluated in approximately 1.8 milliseconds, which is imperceptible to web users.
- **Scalability Pathway to $O(\log N)$:** For $N > 100,000$, we apply Bounding Box SQL filtering: `WHERE location\_lat BETWEEN lat - Δ AND lat + Δ`, which uses standard B-tree database indexes in $O(\log N)$ time, calculating Haversine only on the surviving bounding-box candidates.

Q10: Explain the exact logic of the Anti-Flipping cooldown algorithm.

- **Temporal Delta Calculation:** When a request is submitted, we query the most recent fulfilled request for that user in that category: `fulfilled = Request.query.filter_by(receiver_id=uid, status='Fulfilled').order_by(updated_at.desc()).first()`.
- **Cooldown Check:** We compute `days_passed = (datetime.utcnow() - fulfilled.updated_at).days`. For `Electronics`, if `days_passed < 365`, the request is rejected and the user is told `Cooldown unlocks in (365 - days\_passed) days`. For `Household`, the threshold is 180 days.

Q11: How does your system prevent fake self-donations between friends or on the same device?

- **Dual-Layer IP & Network Check:** Upon fulfillment, the server evaluates `current_user.last_login_ip == receiver.last_login_ip`. If they match (and are not localhost), the transaction is flagged and `points_awarded` is forced to 0.
- **Weekly Velocity Limiter:** Even if bad actors use VPNs to cycle IP addresses, the rolling 7-day query caps points to 300 maximum per week, destroying any incentive for bot-driven point farming.

Q12: Why are Karma points non-transferable?

- **Sybil Attack Mitigation:** If points were transferable, bad actors could create 50 bot accounts, harvest beginner points, and transfer them into a single primary account to unlock Platinum status fraudulently. Non-transferability guarantees that trust scores reflect genuine individual community service.

Q13: What regex pattern validates the NITI Aayog NGO Darpan ID?

- **Regex:** `^[A-Z]{2}\d{4}\d{7}$`
- **Structural Breakdown:** `[A-Z]{2}` represents the 2-letter state code (e.g., `AP`, `MH`, `DL`); `/` is the mandatory delimiter; `\d{4}` is the 4-digit registration year; and `\d{7}` is the 7-digit unique sequential non-profit identifier issued by NITI Aayog.

Q14: How is the community ❤️ Like button toggle made idempotent?

- **Database Constraint:** `NGOLike` has `\_\_table\_args\_\_ = (db.UniqueConstraint('user\_id', 'ngo\_id'),)`.
- **Toggle Logic:** The controller executes `like = NGOLike.query.filter\_by(user\_id=uid, ngo\_id=nid).first()`. If `like` exists, `db.session.delete(like)` is called. If not, `db.session.add(NGOLike(user\_id=uid, ngo\_id=nid))` is called. Because of the unique constraint, no race condition can ever produce duplicate likes for the same user.

Q15: How does the Bcrypt algorithm ensure passwords cannot be cracked with modern GPUs?

- **Memory-Hard Algorithm:** Unlike SHA-256 or MD5 which are fast mathematical hashes designed for message verification, Bcrypt requires 4 KB of high-speed memory for Blowfish S-boxes during key expansion. This memory-hard requirement drastically throttles massively parallel GPU and ASIC hardware crackers.

5.3 Category 3: Database Design, Schema & Concurrency**Q16: Explain how referential integrity and cascading deletes work in your SQLAlchemy models.**

- **Parent-Child Relationship:** In `app/models.py`, `Request` defines `messages = db.relationship('Message', backref='request', cascade='all, delete-orphan')`.
- **Operational Benefit:** If a donor deletes an expired listing or an admin removes a fraudulent request, SQLAlchemy automatically cascades the deletion down to all associated chat messages, preventing orphaned child records from corrupting database foreign key integrity.

Q17: What are ACID properties and how does your database maintain them?

- **Atomicity:** Every fulfillment operation (updating request status, creating history record, awarding Karma points) executes inside a single `db.session.commit()`. If any step fails, `db.session.rollback()` reverts all modifications completely.
- **Consistency:** Enforced via foreign key constraints (`db.ForeignKey('users.id')`) and column data type constraints.
- **Isolation:** Transactions are isolated by the database engine, preventing dirty reads or phantom reads.
- **Durability:** Once `db.session.commit()` returns, changes are written to persistent disk storage.

Q18: Why did you use SQLite for development and how will you transition to PostgreSQL?

- **Development Efficiency:** SQLite requires zero background server configuration, zero port bindings, and lives inside a single portable file (`donation\_tracker.db`), allowing immediate local execution and test reproducibility.
- **PostgreSQL Production Transition:** Because our entire schema is mapped via SQLAlchemy ORM without raw vendor-specific SQL, moving to PostgreSQL requires merely changing `SQLALCHEMY\_DATABASE\_URI = 'postgresql://user:pass@host:5432/dbname'` in `config.py` and running `flask db upgrade`.

Q19: What is the difference between `lazy=True`, `lazy='dynamic'`, and `lazy='joined'` in SQLAlchemy?

- **`lazy=True` (Select Loading):** Loads related child records only when accessed as an attribute (default). Suitable for collections of modest size like a request's messages.
- **`lazy='dynamic'`:** Returns a SQLAlchemy query object instead of a list. Used on `user.liked\_ngos` so we can chain `.count()` or `.filter()` directly on the database engine without loading thousands of like objects into RAM.
- **`lazy='joined'` (Eager Loading):** Emits an inner/left join in the initial SQL query, eliminating the N+1 query problem when displaying items and donors together.

Q20: What is the N+1 Query Problem and how can it be avoided in your application?

- **The Problem:** If you query 100 resources and then loop through them displaying `resource.donor.email`, a naive ORM emits 1 initial query for resources + 100 separate queries for each donor (101 total queries), choking database performance.
- **The Solution:** Using SQLAlchemy eager loading via `Resource.query.options(db.joinedload(Resource.donor)).all()`. This issues a single `LEFT OUTER JOIN` query, reducing database roundtrips from 101 to exactly 1.

Q21: How are user profile images and documents stored in the database?

- **Best Practice Architecture:** Binary file blobs (BLOBs) are NEVER stored directly in database rows, as that causes catastrophic database bloat and slow backups.
- **File System + DB Pointer:** Files are saved to disk under `app/static/uploads/documents/` and `app/static/uploads/resources/`. The database stores only the sanitized relative string path (`VARCHAR(255)`) to the asset.

Q22: What database indexes are essential for Seva Sankalp in a production environment?

- **Key Indexes:** 1) `users(email)`: Unique index for `$O(1)$` login lookups. 2) `resources(status, category)`: Composite index for fast marketplace filtering. 3) `resources(location_lat, location_lng)`: Geospatial index for bounding-box queries. 4) `requests(receiver_id, status)`: Fast evaluation of anti-hoarding and cooldown limits. 5) `ngo_likes(user_id, ngo_id)`: Unique composite index for instant like state lookup.

Q23: Explain the database schema relationship for direct UPI tracking (`UPIDonationClick`).

- **Tracking Mechanism:** When a donor clicks to scan an NGO's UPI QR code, an asynchronous record is logged in `upi_donation_clicks` with `donor_id`, `ngo_id`, and `timestamp`. This provides the NGO and admin with intent metrics without touching sensitive bank credentials.

5.4 Category 4: Web Security, Cryptography & Privacy Compliance**Q24: How does your platform prevent Cross-Site Request Forgery (CSRF)?**

- **CSRF Protection Mechanism:** We implement Flask-WTF / `CSRFProtect`. Every state-modifying form includes a hidden input `<input type='hidden' name='csrf_token' value='{{ csrf_token() }}>`.
- **Cryptographic Validation:** The token is an HMAC-SHA256 signature combining the user's session identifier and server secret key with a timestamp. An external malicious site cannot forge this signed token, blocking cross-origin unauthorized submissions.

Q25: How does Seva Sankalp prevent SQL Injection attacks?

- **SQLAlchemy Parameterization:** All database interactions execute via SQLAlchemy ORM. The ORM translates Python queries into parameterized SQL statements with placeholders (`?` or `:val`). User inputs are transmitted separately from the SQL command structure, rendering injection attacks such as `" OR '1'='1'` entirely inert.

Q26: How does Jinja2 protect against Cross-Site Scripting (XSS)?

- **Context-Aware Auto-Escaping:** By default, Jinja2 automatically escapes all dynamic expressions `{{ variable }}`. Characters with special HTML meaning (`<`, `>`, `&`, `"`, `'`) are converted to their safe HTML entities (`<`, `>`, `&`, `"`, `'`).
- **Strict Script Policy:** We never use the `| safe` filter on unverified user-generated strings, preventing malicious attackers from injecting `<script>` payloads into resource titles or chat messages.

Q27: How does your architecture comply with India's Digital Personal Data Protection (DPDP) Act, 2023?

- **Purpose Limitation:** Personal data (Aadhaar, MeeSeva certificates) is collected strictly for verification and never monetized or shared with third parties.
- **Beneficiary Privacy & Dignity:** Underprivileged recipients upload income certificates for Admin review only. Donors only see a binary 'Verified Low-Income' badge. Financial PDFs are never publicly exposed or downloadable by peers.
- **Right to Erasure:** When an account is deleted, cascaded relationships ensure personal records, documents, and messages are permanently purged.

Q28: What is Directory Traversal and how do you prevent it in document uploads?

- **The Vulnerability:** If an attacker uploads a file named `../../../../etc/passwd` or `../../../../app/routes.py`, a naive file writer would overwrite critical system files outside the upload folder.
- **Our Defense:** We wrap all uploaded filenames in `werkzeug.utils.secure_filename()`, which strips path traversal sequences (`../`), slashes, and control characters, leaving only safe alphanumeric tokens.

Q29: How do you protect against session hijacking and session fixation?

- **Session Regeneration:** Upon successful login, Flask-Login generates a brand new session identifier, invalidating any pre-authentication session tokens (preventing fixation).
- **Cookie Hardening:** Cookies are marked `HttpOnly` (immune to JavaScript retrieval via XSS) and `SameSite=Lax` (immune to CSRF transmission).

Q30: How do you prevent malicious file uploads (e.g., executing a PHP or Python shell)?

- **Multi-Layer Defense:** 1) Extension allowlist: Only `.jpg`, `.jpeg`, `.png`, `.webp`, `.pdf` are permitted. 2) Static serving: Web servers do not execute scripts inside `/static/uploads/`; files are served with MIME-type `application/octet-stream` or image headers. 3) Filenames are hashed with unique timestamps.

Q31: How does your in-app chat ensure end-to-end authorization?

- **Authorization Guard:** In `app/core/request_routes.py`, `get_messages(req_id)` and `send_message(req_id)` explicitly verify: `if current_user.id not in [req.receiver_id, req.resource.donor_id] and current_user.role != 'admin': return 403 Forbidden`. No third party can eavesdrop on handover conversations.

5.5 Category 5: Frontend Engineering & Geospatial Integration**Q32: How does Leaflet.js render map tiles without lagging the client browser?**

- **Tile Slippy Map Architecture:** Leaflet slices the world into 256×256 pixel image tiles. It computes which tile coordinates (x, y, z) intersect the browser viewport and downloads only visible tiles asynchronously using HTTP GET. As the user pans, off-screen tiles are purged from the DOM to conserve mobile memory.

Q33: Why did you use OpenStreetMap instead of Google Maps API?

- **Zero Cost & No Quotas:** Google Maps requires a credit card and charges \$7 per 1,000 requests after a free quota. For a free student/civic project, OpenStreetMap tiles are 100% free and open-source.
- **User Privacy:** OpenStreetMap does not profile or track users across the web, aligning with our privacy-first civic mission.

Q34: Explain how the asynchronous community ❤️ Like button works without refreshing the page.

- **JavaScript Fetch Flow:** 1) User clicks heart button. 2) JavaScript intercepts event, prevents default page navigation. 3) Dispatches `fetch('/like-ngo/' + ngoId, { method: 'POST', headers: { 'X-CSRFToken': token } })`.

4) Server toggles database record and returns `{ liked: true, likes\_count: 42 }`. 5) JavaScript immediately toggles `.active` CSS class and updates inner text count.

Q35: How do you handle dark mode styling without flickering on page reload?

- **CSS Variables & LocalStorage:** Theme variables (`.bg-primary`, `.text-primary`) are defined in `.root` and `[data-theme='dark']`. JavaScript reads `localStorage.getItem('theme')` in the `` tag before DOM rendering begins, setting the attribute instantly and eliminating flash of unstyled content (FOUC).

Q36: How are UPI QR codes rendered dynamically for supported NGOs?

- **Standardized UPI VPA String:** According to NPCI specifications, UPI transactions follow the URI schema: `upi://pay?pa={upi\_id}&pn={ngo\_name}&cu=INR`. We feed this dynamic string into a client-side QR generation script or secure CDN API, rendering a clean, scannable QR code matching any UPI app (Google Pay, PhonePe, Paytm).

Q37: What happens if a user disables browser GPS location permissions?

- **Graceful Degradation:** If `navigator.geolocation` fails or is denied by the user, the map falls back to a default regional center (Vizianagaram / Visakhapatnam coordinates: `[18.11, 83.40]`) and alerts the user to enter their town or pincode manually.

5.6 Category 6: Scalability, Production Deployment & Edge Cases

Q38: What will break first if 100,000 active users join Seva Sankalp tomorrow, and how will you fix it?

- **Bottleneck 1: SQLite File Locking:** SQLite locks the entire database file during writes. At high concurrency, requests will queue and throw `sqlite3.OperationalError: database is locked`. Fix: Migrate immediately to PostgreSQL with connection pooling (PgBouncer).
- **Bottleneck 2: In-Memory Haversine:** Iterating 100,000 items in Python will take ~200 ms per search. Fix: Use PostGIS extension on PostgreSQL with spatial R-tree indexes (`ST\_DWithin`).
- **Bottleneck 3: Synchronous Email Delivery:** Sending OTP emails blocks the worker thread for 1-2 seconds. Fix: Offload email dispatching to Celery with Redis as a message broker.

Q39: How would you implement Redis caching in Seva Sankalp?

- **High-Impact Caching Areas:** 1) Cache `/support-ngos` list in Redis with a 1-hour TTL, since approved NGO lists change infrequently. 2) Cache user leaderboard and Karma scores in a Redis Sorted Set (`ZADD` / `ZREVRANGE`) for $O(\log N)$ instant leaderboard queries.

Q40: How do you deploy Seva Sankalp on a production Linux server using Gunicorn and Nginx?

- **Architecture:** Nginx acts as the reverse proxy on port 80/443 handling SSL termination and static files directly. Gunicorn runs behind Nginx as the WSGI application server with $W = (2 \times \text{CPUs}) + 1$ worker processes communicating via a local Unix socket (`/tmp/sevasankalp.sock`).

Q41: How does the platform handle network drops during chat messaging?

- **Resilient Error Handling:** The client-side AJAX fetch includes a `.catch(error => { showToast('Network connection lost. Message not delivered. '); button.disabled = false; })` block, preventing messages from being lost in limbo without user notification.

Q42: What database backup strategy would you implement for production?

- **Automated Strategy:** 1) Daily automated snapshots using `pg\_dump` with gzip compression piped to secure AWS S3 cold storage. 2) Point-In-Time Recovery (PITR) via Write-Ahead Logging (WAL) archiving, allowing database restoration to any exact second in the past 14 days.

5.7 Category 7: Future Technical Upgrades & Emerging Technologies

Q43: How will production DigiLocker verification be implemented via API Setu?

- **Government Integration:** We will register Seva Sankalp as a verified consumer entity on the Government of India's ****API Setu**** gateway. The user will be redirected to the official DigiLocker consent screen, authenticate via Aadhaar OTP, and API Setu will return a cryptographically signed JSON/XML payload verifying their BPL or MeeSeva Income status directly to our webhook.

Q44: How will direct Google Login be integrated?

- **Google Identity Services (OAuth 2.0):** We will configure OAuth 2.0 client credentials in Google Cloud Console, render the Google One-Tap SDK, receive a signed JWT ID token on the callback route, verify the cryptographic signature using Google's public keys via `google-auth` library, and log the user in automatically.

Q45: How will mediated UPI payments work with Razorpay Webhooks?

- **Payment Gateway Architecture:** Instead of only showing static QR codes, donors can enter custom amounts for NGO relief campaigns. Razorpay generates an order ID, opens standard checkout, and triggers an asynchronous server webhook with a signed HMAC-SHA256 signature upon bank capture. The server verifies the signature, credits the campaign ledger, and automatically generates an 80G tax exemption PDF receipt.

6. QUICK REVISION CHEAT-SHEET (SUMMARY MATRIX)

Use this rapid-fire reference matrix for the 5 minutes immediately before entering the viva defense room:

Parameter / Metric	Technical Specification in Seva Sankalp	Exam / Viva Defense Formula or Reference
Haversine Earth Radius (R)	6,371.0 Kilometers	$d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$; $a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cos \phi_2 \sin^2(\Delta\lambda/2)$
Electronics Cooldown Window	365 Days (12 Months)	<code>(now - last_fulfilled.updated_at).days >= 365</code>
Household Cooldown Window	180 Days (6 Months)	<code>(now - last_fulfilled.updated_at).days >= 180</code>
Anti-Hoarding Request Limit	Maximum 3 Active Pending Requests	<code>Request.query.filter_by(receiver_id=id, status='Pending').count() < 3</code>
Karma Points Awarded	50 Points per Fulfilled Exchange	<code>PointsTransaction(user_id=id, amount=50, type='Earned')</code>
Karma Velocity Cap	Maximum 300 Points per 7 Days	<code>SUM(amount for last 7 days) + 50 <= 300</code>
Same-Network Fraud Check	IP Collision Check (Donor vs Receiver)	<code>IF donor.last_login_ip == receiver.last_login_ip THEN points = 0</code>
Karma Badge Tiers	Bronze: 100 Silver: 500 Gold: 1000 Plat: 5000	<code>current_user.points_balance</code> thresholding
Darpan ID Regex Pattern	14 Characters: 2 letters, 4 year, 7 digits	<code>^[A-Z]{2}\d{4}\d{7}\$</code>
PAN Card Regex Pattern	10 Characters alphanumeric	<code>^[A-Z]{5}[0-9]{4}[A-Z]{1}\$</code> (4th char: C, T, or A)
Password Hashing Algorithm	Bcrypt (Blowfish cipher, salt, cost 12)	<code>bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())</code>

Parameter / Metric	Technical Specification in Seva Sankalp	Exam / Viva Defense Formula or Reference
CSRF Protection Mechanism	HMAC-SHA256 Token Injection	Flask-WTF CSRFProtect middleware on all stateful POST requests
Database Cascade Rule	<code>cascade='all, delete-orphan'</code>	Guarantees no orphaned messages or requests on resource deletion
Leaflet Tile Provider	OpenStreetMap Carto Tile Layer	L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png')
Like Button Idempotency	<code>UniqueConstraint('user_id', 'ngo_id')</code>	Prevents duplicate likes; toggles state dynamically via AJAX